

Guia de Estudo e Apresentacao

Este documento explica o projeto do tradutor de Libras em uma linguagem pensada para apresentacao. A ideia e voce conseguir responder o que cada parte faz, por que cada biblioteca foi usada e qual e a logica completa do sistema.

1. Objetivo do Projeto

O projeto e um tradutor de Libras com IA que reconhece sinais em tempo real usando a webcam.

O sistema captura a imagem da camera, extrai pontos do corpo, maos e rosto com MediaPipe, transforma esses pontos em numeros e envia uma sequencia de frames para um modelo LSTM treinado. O modelo retorna a classe mais provavel, como Oi , Casa , Banheiro , Obrigado , Livro , Nome ou Desconhecido .

Em uma frase:

O projeto transforma movimentos em sequencias numericas e usa aprendizado supervisionado para classificar sinais de Libras.

2. Organizacao Atual do Projeto

```
Projeto_Tradutor_de_Libras/  
|-- app.py  
|-- etapa2_preprocessamento.py  
|-- etapa3_treinamento.py  
|-- front/  
|   |-- templates/  
|   |   |-- index.html  
|   |-- static/  
|       |-- css/style.css  
|       |-- js/app.js  
|-- models/  
|   |-- modelo_libras.keras  
|   |-- label_encoder.pkl  
|   |-- thresholds.pkl  
|-- data/  
|   |-- landmarks.csv  
|   |-- feedback_landmarks.csv  
|-- reports/  
|   |-- resultado_treinamento.png  
|   |-- resultado_loss.png  
|   |-- matriz_confusao.png  
|-- videos/  
    |-- data/
```

Arquivos principais

`app.py` : Backend Flask. Carrega o modelo treinado, recebe frames da webcam, extrai landmarks, faz a predicao e devolve o resultado para o front.

`front/templates/index.html` : Estrutura visual da pagina.

`front/static/css/style.css` : Estilo da interface.

`front/static/js/app.js` : Controla webcam, envia frames para o backend, desenha landmarks, atualiza historico e envia feedback.

`etapa2_preprocessamento.py` : Le os videos do dataset, extrai landmarks com MediaPipe e gera `data/landmarks.csv`.

`etapa3_treinamento.py` : Le os CSVs, treina o modelo LSTM, salva o modelo final e gera graficos de desempenho.

`models/modelo_libras.keras` : Modelo treinado.

`models/label_encoder.pkl` : Mapeia classes de texto para numeros e numeros para texto.

`models/thresholds.pkl` : Guarda limites de confianca calibrados por classe.

3. Bibliotecas Usadas

Flask

O Flask e usado para criar o servidor web em Python.

No projeto ele faz tres coisas principais:

- entrega a pagina do front;
- recebe frames da webcam na rota `/predict` ;
- recebe correcoes do usuario na rota `/feedback` .

Exemplo no projeto:

```
@app.route("/predict", methods=["POST"])
def predict():
    ...
```

Isso significa que o navegador envia uma imagem para o backend, e o Flask chama a funcao `predict()` .

OpenCV

O OpenCV e usado para manipular imagem no backend.

No projeto ele decodifica a imagem enviada pelo navegador em base64:

```
frame = cv2.imdecode(nparr, cv2.IMREAD_COLOR)
```

Tambem redimensiona o frame para deixar o processamento mais leve:

```
frame = cv2.resize(...)
```

MediaPipe

O MediaPipe e a biblioteca que detecta pontos do corpo.

O projeto usa:

- `Hands` : pontos das maos;
- `Pose` : pontos do corpo;
- `FaceMesh` : alguns pontos do rosto.

Esses pontos sao chamados de `landmarks` .

Cada frame vira um vetor de 288 features:

```
Pose:          33 pontos * 4 valores = 132
Mao esquerda: 21 pontos * 3 valores = 63
Mao direita:  21 pontos * 3 valores = 63
Rosto:         10 pontos * 3 valores = 30
Total: 288 features
```

NumPy

NumPy e usado para trabalhar com arrays numericos.

Ele aparece em partes como:

- montar o vetor de features;
- calcular movimento;
- reorganizar os dados no formato esperado pela LSTM;
- calcular medias, diferencas e probabilidades.

Exemplo:

```
X = np.array(list(frame_buffer), dtype=np.float32).reshape(1, 20, 288)
```

Isso cria uma entrada com:

```
1 amostra
20 frames
288 features por frame
```

Pandas

Pandas e usado para ler e criar CSVs.

No preprocessamento:

```
df.to_csv(ARQUIVO_SAIDA, index=False)
```

No treinamento:

```
df = pd.read_csv(ARQUIVO_CSV)
```

Ou seja, o Pandas e a ponte entre os dados extraidos dos videos e o treinamento do modelo.

TensorFlow / Keras

TensorFlow/Keras e usado para criar, treinar e carregar o modelo de IA.

No treinamento, o projeto constroi uma rede LSTM:

```
model = construir_modelo(MAX_FRAMES, N_FEATURES, n_classes)
```

No app, o modelo ja treinado e carregado assim:

```
model = tf.keras.models.load_model(MODEL_PATH)
```

Scikit-learn

Scikit-learn e usado para tarefas auxiliares de machine learning:

- separar treino e teste;
- converter labels em numeros;
- gerar metricas;
- calcular matriz de confusao.

Exemplo:

```
le = LabelEncoder()  
y_int = le.fit_transform(labels)
```

Isso transforma classes como Oi , Casa , Livro em numeros, porque o modelo trabalha com numeros.

Matplotlib

Matplotlib gera graficos do treinamento:

- acuracia;
- loss;
- matriz de confusao.

Esses arquivos ficam em reports/ .

4. Fluxo Completo do Sistema

Fluxo de treinamento

```
videos/data/*.mp4
  ↓
etapa2_preprocessamento.py
  ↓
MediaPipe extrai landmarks
  ↓
data/landmarks.csv
  ↓
etapa3_treinamento.py
  ↓
Treina LSTM
  ↓
models/modelo_libras.keras
models/label_encoder.pkl
models/thresholds.pkl
```

Fluxo em tempo real

```
Webcam no navegador
  ↓
JavaScript captura frame
  ↓
Frame enviado em base64 para Flask
  ↓
app.py decodifica imagem com OpenCV
  ↓
MediaPipe extrai landmarks
  ↓
Sistema guarda sequencia de 20 frames
  ↓
Modelo LSTM prediz probabilidades
  ↓
Backend aplica filtros, votos e thresholds
  ↓
Frontend mostra o sinal detectado
```

5. Por Que Usar Sequencia de 20 Frames?

Um sinal de Libras não é apenas uma foto parada. Ele envolve movimento.

Por isso o modelo recebe uma sequência:

```
MAX_FRAMES = 20  
N_FEATURES = 288
```

Ou seja:

```
20 momentos do movimento  
288 numeros em cada momento
```

Esse formato permite que a LSTM aprenda o movimento ao longo do tempo.

6. Por Que LSTM?

LSTM é um tipo de rede neural recorrente feita para dados sequenciais.

Ela é adequada porque sinais em Libras dependem de ordem temporal:

- onde a mão começa;
- para onde ela vai;
- como o corpo e o rosto acompanham;
- como o gesto termina.

Uma rede comum analisaria os dados de forma mais estática. A LSTM consegue considerar a evolução do movimento.

7. Atenção Temporal no Modelo

O modelo tem uma camada de atenção temporal.

A ideia é permitir que a rede dê mais peso aos frames mais importantes do gesto.

Exemplo: em um sinal, talvez o início seja pouco informativo, mas o movimento final diferencie melhor uma classe da outra.

Então a atenção ajuda o modelo a focar nos momentos mais relevantes da sequência.

8. Normalização dos Landmarks

A normalização tenta reduzir diferenças como:

- pessoa mais perto ou mais longe da câmera;
- corpo mais para esquerda ou direita;
- enquadramento diferente;
- tamanho diferente da pessoa.

Em vez de usar os pontos crus da câmera, o sistema centraliza e escala os pontos com base no corpo ou nos próprios landmarks.

Isso ajuda o modelo a aprender o gesto, e não apenas a posição da pessoa na imagem.

9. Classe Desconhecido

A classe `Desconhecido` foi criada para ensinar o modelo que nem tudo é um sinal válido.

Sem essa classe, o modelo sempre tentaria escolher uma das classes conhecidas, mesmo quando a pessoa estivesse fazendo nada ou fazendo outro gesto.

Com `Desconhecido`, o sistema consegue rejeitar gestos fora do vocabulário.

No app, quando a classe prevista é de rejeição, ela não entra na frase final.

10. Votacao, Threshold e Debounce

O modelo retorna probabilidades, mas o app não aceita qualquer resposta imediatamente.

Ele aplica algumas regras para deixar a detecção mais robusta.

Votacao

O sistema olha uma janela de previsões recentes:

```
JANELA_PREDICOES = 3
VOTOS_NECESSARIOS = 2
```

Isso significa que a classe precisa aparecer de forma consistente antes de ser aceita.

Threshold

Threshold é o mínimo de confiança para aceitar uma classe.

Exemplo:

```
Oi: 0.90
Casa: 0.55
```

Esses valores ficam em `models/thresholds.pkl` e são calibrados no treino.

Margem

Margem é a diferença entre a classe mais provável e a segunda mais provável.

Se o modelo está em dúvida entre duas classes, a margem fica baixa. O app pode rejeitar a resposta.

Debounce

Debounce evita repetir o mesmo sinal várias vezes seguidas.

Depois que um sinal é aceito, o app espera alguns frames antes de aceitar outro.

```
DEBOUNCE_FRAMES = 10
```


11. Feedback Humano

Foi implementado um mecanismo de feedback na interface.

Quando o sistema acerta, o usuario clica em `Acertou`.

Quando erra, o usuario escolhe o label correto e clica em `Corrigir`.

Isso salva uma nova amostra em:

```
data/feedback_landmarks.csv
```

Esse processo e aprendizado supervisionado, porque cada exemplo salvo tem:

```
entrada: landmarks do gesto  
saida correta: label informado pelo usuario
```

Importante:

O modelo nao aprende no clique. O clique salva dados rotulados. O aprendizado acontece quando rodamos `etapa3_treinamento.py` novamente.

12. Por Que Nao Precisa CSV Para Rodar no Dia?

Para rodar o tradutor em tempo real, o app usa o modelo ja treinado:

```
models/modelo_libras.keras  
models/label_encoder.pkl  
models/thresholds.pkl
```

Os CSVs so sao necessarios para retreinar.

Entao, no dia da apresentacao, basta ter:

```
app.py  
front/  
models/  
requirements.txt
```

E rodar:

```
python app.py
```

13. Data Augmentation

Como o dataset era pequeno, o treinamento usa data augmentation.

Isso cria variacoes artificiais dos dados, por exemplo:

- pequeno ruido nos landmarks;
- pequenas mudancas de escala;
- pequenos deslocamentos;
- variacao de velocidade;
- dropout de landmarks, simulando falha de rastreamento.

Isso ajuda o modelo a ficar mais robusto sem gravar muitos videos novos.

14. Relatorios do Treinamento

O treinamento gera arquivos em `reports/`.

`resultado_treinamento.png` : Mostra a evolucao da acuracia.

`resultado_loss.png` : Mostra a perda durante o treino.

`matriz_confusao.png` : Mostra quais classes o modelo esta confundindo.

A matriz de confusao e muito importante para explicar erros. Se `Nome` aparece muitas vezes como `Livro`, por exemplo, significa que esses sinais estao parecidos para o modelo.

15. O Que Foi Melhorado no Projeto

Principais melhorias feitas:

- separacao do front em `front/templates` e `front/static`;
- organizacao de arquivos em `data`, `models` e `reports`;
- classe `Desconhecido` para rejeitar gestos fora do vocabulario;
- feedback humano para melhorar o dataset;
- thresholds calibrados por classe;
- votacao por janela de predicoes;
- debounce para evitar repeticao;
- filtro de movimento;
- tolerancia quando o MediaPipe perde a mao por poucos frames;
- data augmentation mais realista;
- graficos e matriz de confusao;
- modelo treinado salvo no Git para rodar sem retreinar.

16. Possiveis Perguntas do Professor

Por que usar MediaPipe?

Porque ele ja fornece landmarks confiaveis de maos, corpo e rosto. Assim o modelo nao precisa aprender diretamente a partir dos pixels da imagem, o que exigiria muito mais dados.

Por que usar LSTM?

Porque sinais são movimentos. A LSTM trabalha bem com sequências e consegue considerar a ordem dos frames.

O que são as 288 features?

São os valores numéricos extraídos dos landmarks em cada frame:

```
132 da pose
63 da mão esquerda
63 da mão direita
30 do rosto
```

O sistema aprende em tempo real?

Não exatamente. Ele coleta feedback em tempo real, mas o aprendizado acontece quando o treinamento é executado novamente.

Que tipo de aprendizado é usado?

Aprendizado supervisionado, porque o modelo aprende com exemplos rotulados.

Por que existe a classe Desconhecido?

Para evitar que o modelo classifique qualquer movimento como um sinal conhecido. Ela funciona como uma classe de rejeição.

Por que os CSVs não estão no Git?

Porque são dados gerados e podem ser grandes. Para rodar a aplicação, basta ter o modelo treinado. Os CSVs só são necessários para retreinar.

O que acontece se a iluminação mudar?

Pode afetar a detecção, mas como usamos landmarks do MediaPipe e normalização, o sistema fica menos dependente dos pixels crus. Ainda assim, ambiente e câmera influenciam.

Por que usar thresholds por classe?

Porque algumas classes são mais fáceis de reconhecer que outras. Um único limite global pode ser injusto. Threshold por classe permite calibrar melhor cada sinal.

17. Comandos Importantes

Rodar o app:

```
python app.py
```

Gerar CSV a partir dos vídeos:

```
python etapa2_preprocessamento.py
```

Treinar novamente:

```
python etapa3_treinamento.py
```

Instalar dependencias:

```
pip install -r requirements.txt
```

18. Resumo Final

O projeto usa visao computacional para transformar imagens em landmarks e aprendizado supervisionado para classificar sinais. A parte mais importante e que o modelo nao ve a imagem diretamente; ele aprende a partir dos pontos extraidos pelo MediaPipe. Isso torna o treinamento mais viavel com poucos dados e permite rodar em tempo real com webcam.